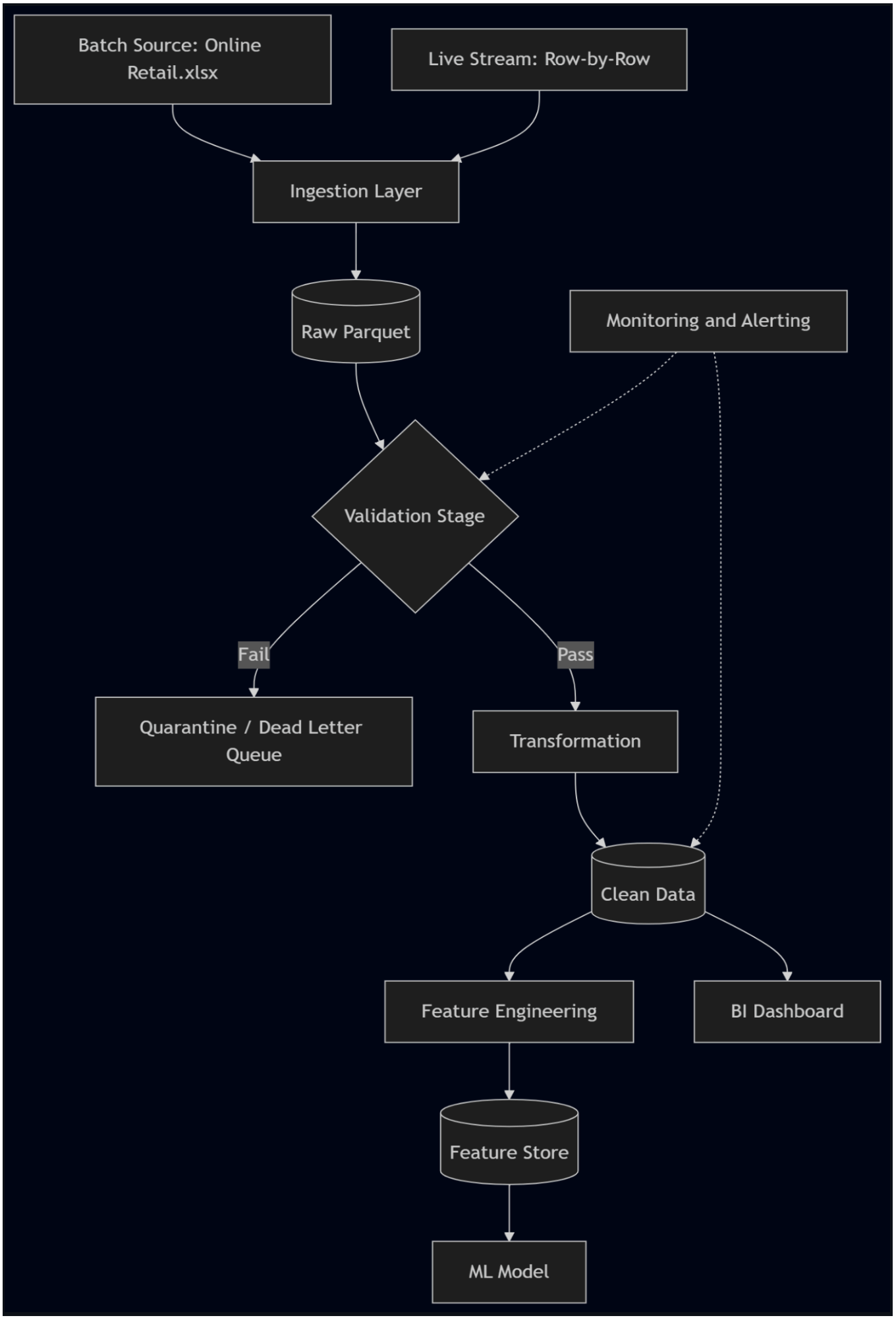
Task 1: Pipeline Architecture Diagram



Batch Source: The historical data comes from the Online Retail.xlsx file. It is loaded once and sent to the ingestion layer.

Live Stream: New transactions arrive one row at a time. Each new order is sent to the ingestion layer in real time.

Ingestion Layer: Receives data from both sources. It reads the data and sends it to raw storage.

Raw Parquet: All incoming records are stored here exactly as received. Nothing is changed or deleted at this stage.

Validation Stage: Every record is checked against schema, value range and business rules. Records that pass go to transformation, records that fail go to quarantine.

Quarantine / Dead Letter Queue: Failed records are stored here for review. The team is notified and decides whether to fix and resubmit them.

Transformation: Valid records are cleaned and new columns are added, such as LineTotal and is_cancellation.

Clean Data: Stores all validated and transformed records. The BI dashboard reads directly from this layer.

Feature Engineering: Customer level features such as total revenue and order count are calculated here for the ML model.

Feature Store: Stores the final features. Updated daily and kept for 90 days.

BI Dashboard: Reads from the clean layer to show daily sales and country level reports.

ML Model: Reads from the feature store and is retrained weekly to predict high value customers.

Monitoring and Alerting: Checks the pipeline after each stage. Send alerts if something goes wrong.

Task 2: Validation and Error Handling Design

2.1 – Define validation rules

A-Schema validations (structural correctness):

Rule 1: InvoiceNo should be str format and must be not empty.
Rule 2: StockCode - String format again and also non-empty
Rule 3: Description - String format and may be empty
Rule 4: Quantity- integer format and cannot be null and non-zero
Rule 5: InvoiceDate - datetime and not null
Rule 6: UnitPrice - float and not empty
Rule 7: CustomerID - Integer and must be non-empty
Rule 8: Country - String and non-empty

B-Value range validations (sensible values):

Rule 1: Quantity- for non-cancelaration must be > 0 , Quantity for cancellation records must be < 0.
Rule 2: Quantity- must be within the range [-10,000, 10,000]. Values outside this range are treated as data entry errors.
Rule 3: UnitPrice - must be >= 0.0. Negative prices are invalid

C-Business rule validations (domain logic):

Rule 1: Quantity- If InvoiceNo starts with 'C', then Quantity must be negative
Rule 2: If Quantity is negative and InvoiceNo does not start with 'C', this is an inconsistency — the record is quarantined.
Rule 3: Fractional or negative CustomerIDs are invalid.

2.2– Design the error handling flow

For category A:

- Record rejected entirely
- Written to quarantine table as Parquet, with appended fields: rule and reason
- If the error rate in a batch goes above 1%, the data engineering team should be informed so they can investigate the issue.
- After notification, the problematic records must be corrected and resubmitted through the pipeline. This time they go through the same validation checks again — if they pass, they are moved to the clean layer, if not, they stay in quarantine for further review.

For category B:

- Records are rejected and quarantined for review.
- Written to quarantine table
- Summary report is generated listing the count and percentage of range failures per rule, and sending to data team
- If the problem is a known bug, the team fixes it and resubmits the affected records.

For category C:

- Records are rejected and quarantined for review.
- Written to quarantine table
- Any business rule failure is reported to the data engineering team immediately.
- The team reviews the quarantined records and decides whether the data is valid or not. Valid records are resubmitted, invalid ones are discarded.

Task 3: Transformation and Storage Design

3.1 – Define transformations

Transformation 1:

Input: String fields from the clean layer.

Output: Same fields,leading and trailing whitespace is removed from all text fields

Idempotency: Yes — applying strip() twice produces the same result.

Transformation 2:

Input: Quantity and UnitPrice .

Output: New column LineTotal = Quantity * UnitPrice.

Idempotency: Yes — same result.

Transformation 3:

Input: InvoiceNo field.

Output: New boolean column `is_cancellation = True` if InvoiceNo starts with 'C', else `False`.

Idempotency: Yes.

Transformation 4:

Input: InvoiceDate (datetime).

Output: New columns: InvoiceYear , InvoiceMonth , InvoiceDayOfWeek.

Idempotency: Yes — taken from a single datetime field.

Transformation 5:

Input: All clean records grouped by CustomerID.

Output: One row per CustomerID with:

- TotalRevenue — sum of LineTotal for non-cancellation records
- TotalOrders — count of distinct InvoiceNo values
- AvgOrderValue — TotalRevenue / TotalOrders

- UniqueProducts — count of distinct StockCode values purchased

Idempotency: Yes — given the same input records and the same run date, output is identical.

Transformation 6:

Input: Clean records and the outputs from Transformation 5.

Output: RFM metrics for each customer, including:

- Recency: Number of days since the customer's last purchase.
- Frequency: Total number of unique orders placed.
- Monetary**:** Total amount spent by the customer.

Idempotency: Yes.

3.2–Design the storage layers

Layer	Contents	Format	Update frequency	Retention
Raw	All ingested records,same as received.	Parquet	Stream/Batch	Never deleted
Clean	Validated	Parquet	Stream/Batch	Indefinite
Feature	Customer and product features are summarized in one row per entity and updated daily.	Parquet	Daily	90 days

All three layers use Parquet format. Parquet is columnar, which makes reading and filtering data much faster compared to CSV or JSON. It also takes less storage space and works with tool like pandas.

The raw layer is never modified, so Parquet is a good fit since it is designed for write-once storage. The clean layer is partitioned by date, which allows queries to skip unnecessary data and run faster. The feature layer stores 90 days of snapshots so the ML team can train on historical data and track changes over time.

3.3 – Design incremental updates

The pipeline keeps a record of which data has already been processed. For batch data, the last processed file is saved. For stream data, the last read message position is saved. This way, the pipeline does not process the same data twice.Sometimes records arrive after their partition was already processed. In this case, they are added to the correct partition with a late flag.When a new transaction arrives through the stream, that customer's features are also updated immediately so the ML model always has fresh data.